SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 – ASSESSMENT TOOL 2

Scalability Audit of an E-Commerce Platform

Name : Natesa krishnanV

Reg.no : 192511311

Course Code : CSA0617

Given Information

ShopSphere has n = 100,000 products currently and is expected to reach n = 1,000,000. The server executes 10 basic operations per second. The nightly processing window is 10 minutes = 600 seconds, allowing 10 × 600 = 6,000 basic operations under the stated model.

(a) Understand the Problem

Module	Input size	Basic operation	Expected growth
M1 – Duplicate-Listing Detector	n products	Compare each product with every later product	$\Theta(n^2)$
M2 – API Rate Limiter	n requests	Audit(Request[j])	$\Theta(n)$
M3 S1 – Sequential Search	n products	Compare search key with each product	$\Theta(n)$ worst case
M3 S2 – Binary Search	n sorted entries	Compare key with middle entry	$\Theta(\log n)$
M4 – Recommendation Engine	n records	Recursive partitioning and merging	Current $\Theta(n^2)$; proposed $\Theta(n \log n)$
M5 – Sales Analytics Engine	n sales records	Recursive processing with n log n work per level	$\Theta(n \log^2 n)$

M1 examines pairs of products. M2 processes windows whose sizes double. S1 scans sequentially, while S2 discards half of the sorted search range after each comparison. M4's recurrence depends on the number of recursive partitions. M5 performs n log n non-recursive work at each recursion level.

(b) Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

Pseudocode:

```
FOR i ← 0 TO n − 2
  FOR j ← i + 1 TO n − 1
    IF Product[i].title = Product[j].title AND
      Product[i].seller = Product[j].seller
       MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

For i = 0, there are n − 1 comparisons; for i = 1, n − 2; continuing down to 1 comparison.

$T(n) = (n - 1) + (n - 2) + \ldots + 1 = \Sigma(k = 1 \text{ to } n - 1)\ k = n(n - 1)/2$.

Exact comparisons: $T(n) = (n^2 - n)/2$. Time complexity is $\Theta(n^2)$. Auxiliary space is $\Theta(1)$, excluding input storage and any storage used for duplicate records or logs.

M2 – API Rate Limiter

Pseudocode:

```
i ← 1
WHILE i < n
  FOR j ← 0 TO i − 1
    Audit(Request[j])
  END FOR
  i ← i × 2
END WHILE
```

The audit counts are $1 + 2 + 4 + \ldots + 2^{(m-1)}$, where $m = \text{ceil}(\log_2 n)$. By the geometric-series formula, $T(n) = 2^m - 1 = 2^{\text{ceil}(\log_2 n)} - 1$.

Since $n \leq 2^{\text{ceil}(\log_2 n)} < 2n$ for positive integer n, the count is $\Theta(n)$, not $\Theta(n \log n)$. The team member's claim is incorrect.

For n = 1,024: $T(n) = 2^{10} - 1 = 1{,}023$ audit operations.

(c) Solve a Recurrence by Substitution – S2

Given $T(n) = T(n/2) + 1$, with $T(1) = 1$.

First expansion: $T(n) = T(n/2) + 1$.

Second expansion: $T(n) = T(n/4) + 2$.

Third expansion: $T(n) = T(n/8) + 3$.

After k expansions: $T(n) = T(n/2^k) + k$.

Stop when $n/2^k = 1$, so $k = \log_2 n$ (for powers of two).

Since $T(1) = 1$, $T(n) = 1 + \log_2 n$ for powers of two. Therefore, time complexity is $\Theta(\log n)$.

Implementation	**Time complexity**	**Auxiliary space**
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$, due to call stack

For non-power-of-two sizes, an integer-size convention is needed for the exact recurrence, but the asymptotic bound remains $\Theta(\log n)$.

(d) Apply the Master Theorem

M4 – Current design

$T(n) = 4T(n/2) + n$

Parameter	Value
a	4
b	2
f(n)	n
n^(log_b a)	$n^{(\log_2 4)} = n^2$
Case	Case 1: f(n) is polynomially smaller than n^2
Result	$T(n) = \Theta(n^2)$

M4 – Proposed redesign

$T(n) = 2T(n/2) + n$

Parameter	Value
a	2
b	2
f(n)	n
n^(log_b a)	$n^{(\log_2 2)} = n$
Case	Case 2: $f(n) = \Theta(n^{(\log_b a)})$
Result	$T(n) = \Theta(n \log n)$

When n doubles, quadratic work grows by approximately 4 times. For n log n, the ratio is $2 \log_2(2n)/\log_2 n$, slightly greater than 2. Removing replication greatly improves scalability.

M5 – Sales Analytics Engine

$T(n) = 2T(n/2) + n \log n$. Here $a = 2$, $b = 2$, $f(n) = n \log n$, and $n^{(\log_b a)} = n$.

The basic three Master Theorem cases do not directly cover this logarithmic factor. By the extended Master Theorem, $f(n) = \Theta(n^{(\log_b a)} \log^k n)$ with $k = 1$, so $T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n) = \Theta(n \log^2 n)$.

Recursion-tree check: at level i, total non-recursive work is $n(\log_2 n - i)$. Summing over $\Theta(\log n)$ levels yields $\Theta(n \log^2 n)$.

(e) Analyze Best, Average and Worst Cases

S1 – Sequential Search

Case	Input condition	Comparisons
Best	Product is first	1
Average, if present	Position uniformly distributed	(n + 1)/2
Worst, if present	Product is last	n
Worst, if absent	Product does not exist	n

If the product exists with probability p and otherwise the full list is scanned:

$E[T(n)] = p(n + 1)/2 + (1 - p)n = n - p(n - 1)/2$.

For n = 1,000,000 and p = 0.5: $E[T(n)] = 0.5(1{,}000{,}001/2) + 0.5(1{,}000{,}000) = 750{,}000.25$ comparisons, approximately 750,000.

S2 – Binary Search

Case	Input condition	Comparisons
Best	Key equals middle entry	1
Average	Key found after typical halvings	$\Theta(\log n)$
Worst	Deepest level or absent key	At most $\text{floor}(\log_2 n) + 1$ for conventional binary search

For n = 1,000,000, $\text{floor}(\log_2 n) + 1 = 20$. Binary search therefore needs at most about 20 comparisons if the index is sorted.

(f) Prove an Asymptotic Bound

Given $f(n) = 3n^2 + 10n \log_2 n + 50$.

Upper bound

For $n \geq 2$, $\log_2 n \leq n$ and $1 \leq n^2$. Thus $f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$. Choose $c_2 = 63$.

Lower bound

For $n \geq 2$, all terms are non-negative, so $f(n) \geq 3n^2$. Choose $c_1 = 3$.

Formal Θ proof

With $c_1 = 3$, $c_2 = 63$, and $n_0 = 2$, $0 \leq 3n^2 \leq f(n) \leq 63n^2$ for all $n \geq n_0$. Hence $f(n) = \Theta(n^2)$.

Show that f(n) is not O(n)

Suppose $f(n) = O(n)$. Then $f(n) \leq cn$ for some fixed $c > 0$ for all sufficiently large n. But $f(n) \geq 3n^2$, implying $3n^2 \leq cn$ and $3n \leq c$. This is impossible for arbitrarily large n. Therefore f(n) is not O(n).

(g) Scalability Decision

At 10 operations per second, a 600-second window allows 6,000 basic operations.

Module	Operations / estimate	Estimated time	Within 10 min?
M1 – Duplicate detector	499,999,500,000	49,999,950,000.00 seconds	No
M2 – API rate limiter	1,048,575	104,857.50 seconds	No
S2 – Binary search	At most 20 comparisons	About 2 seconds at stated rate	Yes
M4 current (order estimate)	$\Theta(n^2)$, order of 10^{12}	Order of 3,171 years	No
M4 proposed (order estimate)	$n \log_2 n \approx 19{,}931{,}569$	1,993,156.86 seconds	No
M5 (order estimate)	$n(\log_2 n)^2 \approx 397{,}267{,}426$	39,726,742.56 seconds	No

M1 exact count: $n(n - 1)/2 = 499{,}999{,}500{,}000$ comparisons. M2 exact count: 2^ceil($\log_2 n$) − 1 = 1,048,575 audits. M4 and M5 are asymptotic growth estimates only; exact counts cannot be derived without specified base-case and implementation costs.

Recommendations

- M1: Replace pairwise comparison with hash-based grouping by (seller, title). Expected time can fall from $\Theta(n^2)$ to $\Theta(n)$, assuming effective hashing.
- M2: Use a token bucket or sliding-window counter with bounded work per request; the provided routine exceeds the budget at n = 1,000,000.
- M4: Adopt the non-replicated redesign, reducing $\Theta(n^2)$ to $\Theta(n \log n)$. Consider incremental recommendations or parallel processing if the deadline still cannot be met.
- M5: Avoid repeated sorting and redundant recursive work. Sort once where possible, cache intermediate results, and use incremental aggregation.
- S2: Retain binary search for lookup, with an efficiently maintained sorted index.

Conclusion: M1, M2, M4, and M5 require redesign or further optimization to satisfy the ten-minute target under the stated operation-rate model. S2 is the most scalable search method for a large sorted catalogue.

Appendix: Python Code

This program calculates exact M1/M2 counts and demonstrates sequential and binary search.

```
import math


def m1_comparisons(n):
    return n * (n - 1) // 2
def m2_audits(n):
    total = 0
    i = 1
    while i < n:
        total += i
        i *= 2
    return total



def sequential_search(items, key):
    comparisons = 0
    for i, item in enumerate(items):
        comparisons += 1
        if item == key:
            return i, comparisons
    return -1, comparisons



def binary_search(items, key):
    low, high = 0, len(items) - 1
```

```
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if items[mid] == key:
            return mid, comparisons
        elif items[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1, comparisons


def scalability_estimates(n, rate=10):
    log_n = math.log2(n)
    estimates = [
        ("M1 exact", m1_comparisons(n)),
        ("M2 exact", m2_audits(n)),
        ("S2 upper bound", math.floor(log_n) + 1),
        ("M4 current order estimate", n ** 2),
        ("M4 proposed order estimate", n * log_n),
        ("M5 order estimate", n * log_n ** 2),
    ]

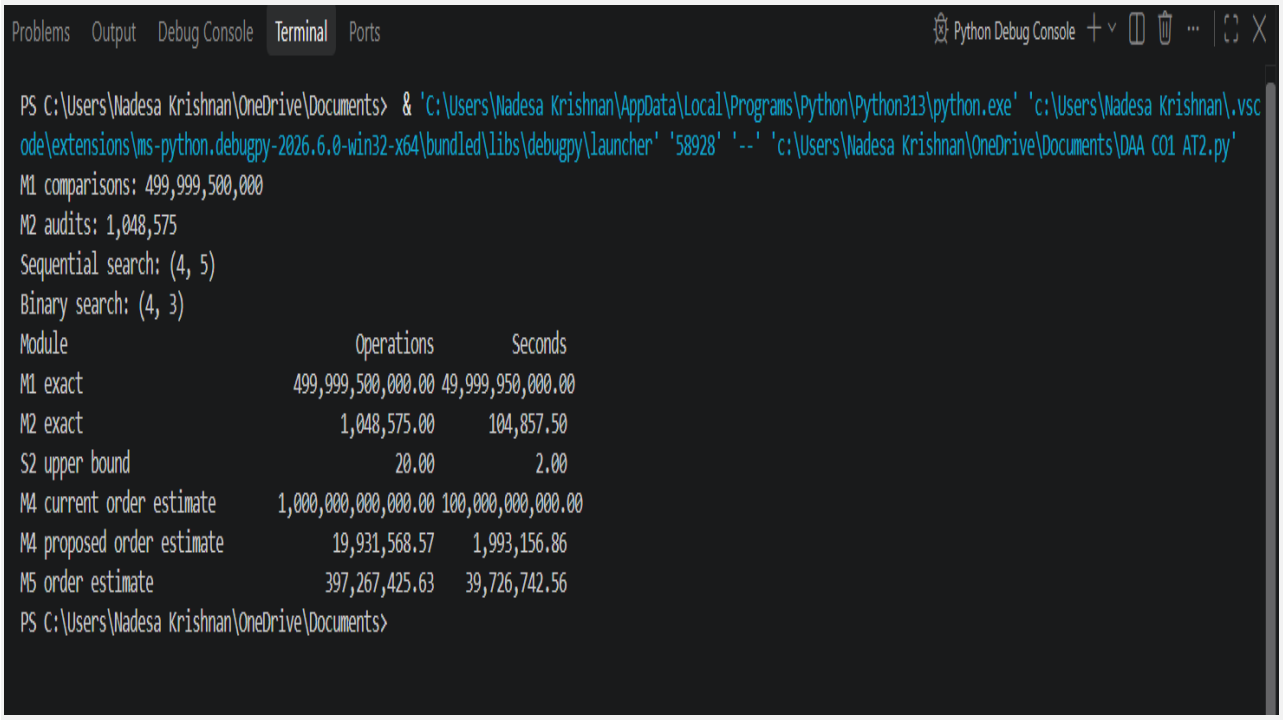
    print(f"{'Module':32} {'Operations':>20} {'Seconds':>16}")
    for name, operations in estimates:
        print(f"{name:32} {operations:20,.2f} {operations / rate:16,.2f}")


if __name__ == "__main__":
    n = 1_000_000
    print("M1 comparisons:", f"{m1_comparisons(n):,}")
    print("M2 audits:", f"{m2_audits(n):,}")

    products = [10, 20, 30, 40, 50, 60, 70, 80]
    print("Sequential search:", sequential_search(products, 50))
    print("Binary search:", binary_search(products, 50))

    scalability_estimates(n)
```

Output:


Problems Output Debug Console Terminal Ports
Python Debug Console
PS C:\Users\Nadesa Krishnan\OneDrive\Documents> & 'C:\Users\Nadesa Krishnan\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Nadesa Krishnan\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58928' '--' 'c:\Users\Nadesa Krishnan\OneDrive\Documents\DAA C01 AT2.py'
M1 comparisons: 499,999,500,000
M2 audits: 1,048,575
Sequential search: (4, 5)
Binary search: (4, 3)
Module Operations Seconds
M1 exact 499,999,500,000.00 49,999,950,000.00
M2 exact 1,048,575.00 104,857.50
S2 upper bound 20.00 2.00
M4 current order estimate 1,000,000,000,000.00 100,000,000,000.00
M4 proposed order estimate 19,931,568.57 1,993,156.86
M5 order estimate 397,267,425.63 39,726,742.56
PS C:\Users\Nadesa Krishnan\OneDrive\Documents>